Module 3: Selections

Sarah Foss

Some slides adapted for use from Dr. Ramon Lawrence

Objectives

- Declare **`boolean`** variables and write Boolean expressions using relational operators.
- Implement decision control with **`if`** and **`switch`** statements.
- Combine conditions using logical operators (**&&**, **||**, **^**, **!**) and write conditional expressions.
- Avoid common errors in **`if`** statements and effectively use decision statements with combined conditions.

Making Decisions

Decisions *(or* ***selections****)* allow the program to perform different actions in certain conditions.

For example, if a person applies for a driver's license and is not 16, then the computer should not give them a license.

To make a decision in a program we must:

1) Determine the ***condition*** in which to make the decision.

   In the license example, we will not give a license if the person is under 16.

2) Tell the computer what actions to take if the condition is true or false.

   A decision always has a *Boolean* or true/false answer.

The syntax for a decision uses the **`if`** statement.

Relational Operators

Relational operators (or **comparison operators**) are used to compare two values or variables

Java Operator	Math Symbol	Name	Example (x is 5)	Result
`<`	<	less than	`x < 0`	`false`
`<=`	≤	less than or equals to	`x <= 0`	`false`
`>`	>	greater than	`x > 0`	`true`
`>=`	≥	greater than or equals to	`x >= 0`	`true`
`==`	=	equal to	`x == 0`	`false`
`!=`	≠	not equal to	`x != 0`	`true`

The result of a comparison expression is either **`true`** or **`false`**
These values are known as **Boolean** values:

```
int age = 10;
boolean result = age > 20;
```

Comparison Example

```
int j = 25;
int k = 45;
double d = 2.5;
double e = 2.51;

// Determine if these comparisons are
true or false

(j == k)                    // false
(j <= k)                    // ??
(d == e)                    // ??
(d != e)                    // ??
(k >= 25)                   // ??
(25 == j)                   // ??
(j > k)                             // ??
(e < d)                             // ??

 j = k;
(j == k)                    // ??
```

`if` Statement

To make decisions with conditions, we use the **`if`** statement.

- If the condition is true, the statement(s) after **`if`** are executed otherwise, they are skipped.
- If there is an **`else`** clause, statements after **`else`** are executed if the condition is false.

Syntax:

```
if (boolean-expression)
   statement;
```

OR

```
if (boolean-expression){
   statement;
} else {
   statement;
}
```

Example:

```
if (age > 17){
  System.out.println("Adult!");
}
```

OR

```
if (age > 17){
   System.out.println("Adult!");
} else {
   System.out.println("Kid!");
}
```

`if` Statement Example

```
Scanner input = new Scanner(System.in);
int age;
boolean teenager;
boolean hasLicense;
System.out.print("Please enter your age: ");
age = input.nextInt();

if (age > 19){
        teenager = false;
        hasLicense = true;
}else{
        if (age < 13){
                teenager = false;
                hasLicense = false;
        }else {
                teenager = true;
                hasLicense = false;
        }
}
System.out.println("Is teenager: " + teenager);
System.out.println("Has license? " + hasLicense);
```

Nested `if` and `else if` statements

Use the **`else if`** clause to specify a new condition if the first condition is **`false`**.

```
if (score >= 80.0){
  System.out.print("A");
} else {
  if (score >= 68.0){
    System.out.print("B");
  } else {
    if (score >= 55.0){
      System.out.print("C");
    } else {
      if (score >= 50.0){
        System.out.print("D");
      } else {
        System.out.print("F");
      }
    }
  }
}
```

```
if (score >= 80.0){
  System.out.print("A");
} else if (score >= 68.0){
  System.out.print("B");
} else if (score >= 55.0){
  System.out.print("C");
} else if (score >= 50.0){
  System.out.print("D");
} else {
  System.out.print("F");
}
```

Curly Braces

You can skip using braces if there is only one statement to execute in the **`if`** statement.

```
int x = 3;
int y = 5;
if (x < y)
        System.out.println("X is less than Y");
```

Note: The else clause always pairs with the most recent **`if`** statement within the same block.

This means that even if you omit braces, you must be careful with how the **`else`** clause is structured to avoid confusion.

Dangling **`else`** Problem

The ***dangling `else` problem*** occurs when a programmer mistakes an **`else`** clause to belong to a different **`if`** statement than it really does.

Brackets determine which statements are grouped together, not indentation! By default, an **`else`** with no brackets matches the closest **`if`** statement regardless of indentation.

Example:

Incorrect

```
if (country == "US"))
   if (state == "HI"))
      shipping = 10.00;
else  // Belongs to 2nd if!
   shipping = 20.00; // Wrong!
```

Correct

```
if (country == "US"){
  if (state == "HI")
      shipping = 10.00;
}else{
   shipping = 20.00;
}
```

Warning!

Another common mistake is adding a semicolon at the end of an if statement

```
if (number > 0);
{
    System.out.println("Positive.");
}
```

This mistake is difficult to find because it doesn't cause a compilation or runtime error.

Instead, it's a logic error, meaning the code block after the if statement will always run, regardless of the condition.

This error often happens when using a next-line block style.

Be Concise!

1)

```
if(number % 2 == 0)
        boolean even = true;
else
        boolean even = false;
```

VS

```
boolean even = number % 2 == 0;
```

2)

```
if(even == true)
        System.out.println("Even!");
```

VS

```
if(even)
        System.out.println("Even!");
```

In both examples, the first and second version accomplish the same goal, but the second versions are more concise and easier to read.

Boolean Expressions

A ***Boolean expression*** is a sequence of conditions combined using AND (**&&**), OR (**||**), XOR(**^**) and NOT (**!**).

Syntax:

(expr1) **&&** *(expr2)* - expr1 AND expr2 :

both must be true for the expression to evaluate to true

(expr1) **||** *(expr2)* - expr1 OR expr2 :

one (or both) must be true for the expression to evaluate to true

(expr1) **^** *(expr2)* - expr1 XOR expr2 :

one (and only one) must be true for the expression to evaluate to true

!*(expr1)* - NOT expr1 :

!(true) == false, !(false) == true

Boolean Expression Examples

```
boolean a, b, c, d, e;
int x = 15;
int num1 = 1;
int num2 = 3;
int month = 10;
int day = 31;

a = (x > 10) && !(x < 50);
b = (month == 9) || (month == 10) || (month == 11);
c = (day == 31 && month == 10);
d = (num1 == 1 ^ num2 == 3);
e = ((10 > 5 || 5 > 10) && ( 10 > 5 && 5 > 10));
```

Short-Circuit Operators (&& and ||)

Short-circuit operators are logical operators (&& and ||) that stop evaluating expressions as soon as the result is determined.

The && operator only evaluates the second expression if the first one is **`true`**.

```
int a = 5, b = 0;
if (b != 0 && a/b != 3) { // This won't cause an error
      System.out.println(a / b);
}
```

The || operator only evaluates the second expression if the first one is **`false`**.

```
int x = 10;
if (x > 0 || x < -10) {
      System.out.println("Condition is true");
}
```

`switch` Statements

A **switch** statement executes one block of code from many possible options based on the value of an expression.

Syntax:

```
switch (expression) {
    case value1:
        // code to be executed if expression equals value1
        break;
    case value2:
        // code to be executed if expression equals value2
        break;
    case value3:
        // code to be executed if expression equals value3
        break;
    // more cases can be added as needed
    default:
        // code to be executed if expression doesn't match any case
}
```

It’s an alternative to using multiple **`if-else`** statements when dealing with several specific values.

`switch` Statements

expression:
The switch expression is evaluated once. It can be of type **`int`**, **`byte`**, **`short`**, **`char`**, **`String`** or **`enum`**

```
int day = 2;
switch (day) {
        case 0:
                System.out.println("Sunday");
                break;
        case 1:
                System.out.println("Monday");
                break;
        case 2:
                System.out.println("Tuesday");
                break;
        case 3:
                System.out.println("Wednesday");
                break;
        default:
                System.out.println("Invalid day");
}
```

`switch` Statements

```
int day = 2;
switch (day) {
        case 0:
                System.out.println("Sunday");
                break;
        case 1:
                System.out.println("Monday");
                break;
        case 2:
                System.out.println("Tuesday");
                break;
        case 3:
                System.out.println("Wednesday");
                break;
         default:
                System.out.println("Invalid day");
}
```

`case:`
Each **`case`** checks if the expression matches a specific value. If it does, the code in that **`case`** block is executed. You cannot use conditions in cases.

`switch` Statements

```
int day = 2;
switch (day) {
        case 0:
                System.out.println("Sunday");
                break;
        case 1:
                System.out.println("Monday");
                break;
        case 2:
                System.out.println("Tuesday");
                break;
        case 3:
                System.out.println("Wednesday");
                break;
        default:
                System.out.println("Invalid day");
}
```

`break`:
After a case is executed, the **`break`** statement is used to exit the **`switch`** block. Without **`break`**, execution would "fall through" and continue into the next case, even if it doesn't match.

`switch` Statements

```
int day = 2;
switch (day) {
        case 0:
                System.out.println("Sunday");
                break;
        case 1:
                System.out.println("Monday");
                break;
        case 2:
                System.out.println("Tuesday");
                break;
        case 3:
                System.out.println("Wednesday");
                break;
        default:
                System.out.println("Invalid day");
}
```

`default:`
This block is optional and is executed if none of the **`case`** values match the expression. It's similar to the **`else`** in an **`if-else`** statement.

Ternary Conditional Operator :?

The **ternary conditional operator (:?**), which consists of three operands, is a short-hand form of the **`if-else`** statement that also returns a value.

Syntax:

```
(boolean-expression) ? expression1 : expression2
```

```
if (x > 0) {
  y = 1;
} else {
  y = -1;
}
```

can be condensed to:

```
y = (x > 0) ? 1 : -1;
```

Ternary Operator Examples

```
int number = 5;
System.out.println((number % 2 == 0) ? "Even" : "Odd");
```

```
int a = 10;
int b = 20;
int max = (a > b) ? a : b;
```

Summary

- The **`boolean`** datatype stores either a **`true`** or **`false`** value.
- Relational operators (**`<`**, **`<=`**, **`==`**, **`!=`**, **`>`**, **`>=`**) return a **`boolean`** result.
- Selection statements, like **`if-else`**, **`switch`**, and the **`ternary`** operator, help make decisions in code.
- If statements decide what code to execute based on whether a condition is **`true`** or **`false`** .
- Boolean operators (**`&&`**, **`||`**, **`!`**, **`^`**) work with **`true`**/**`false`** values. The **`&&`** and **`||`** operators stop evaluating when the result is already known (short-circuiting).
- The switch statement makes decisions based on values of **`char`**, **`byte`**, **`short`**, **`int`**, **`String`**, or **`enum`**.
- The **`break`** keyword in a switch statement prevents execution from falling into the next case.
- The ternary operator **`?:`** is a shorthand for **`if-else`** that returns a value.